LLM Prompts Outside the Translation Benchmark

This document records the model instructions used by the experiment pipeline outside `prompts/benchmark/`. The benchmark translation prompts and their paper origins remain documented in `prompts/benchmark/README.md` and in the benchmark methodology.

The templates below are the core text sent by OxCidation. Values in braces are replaced at runtime. For structured calls, the provider also receives the output schema listed after the prompt. If strict structured output is unavailable, LangChain prepends schema-derived JSON formatting instructions.

Prompt Inventory

Operation	Responsible agent	When it runs	Implementation
Default initial translation	Translator	Main pipeline runs without an explicit benchmark prompt	`prompts/direct_translation_prompt.txt`
Visible-test generation	Tester	Once per selected C program, before translation prompts	`prompts/visible_test_generation_prompt.txt`
Visible-test replacement	Tester, using deterministic and Validator feedback	One bounded round after deterministic rejection or Validator invalidation	Appended in `src/server.py`
Visible-suite correctness review	Validator	After deterministic candidate filtering and before translation	`prompts/visible_test_review_prompt.txt`
Translation-discrepancy analysis	Validator	After C passes a visible case and Rust fails it	Built in `src/server.py`
Rust repair	Translator	After compilation failure or a confirmed translation discrepancy	Built in `src/server.py`

1. Default Initial Translation

The benchmark normally supplies one template from `prompts/benchmark/`. This fallback is used only when no explicit translation prompt is supplied. Its text is identical to the benchmark control `direct-v1`.

```
Your task is only to translate C code to Rust code.
It should be a direct translation of the C code, so before producing the final Rust
code, verify internally:
1. Same input format?
2. Same output format?
3. Same branches?
4. Same loop bounds?
5. Same arithmetic formulas?
6. Same indexing behavior?
7. Same algorithm and data representation?
8. No optimizations or redesigns?
9. Same logging?

C Code:
{c_code}
```

The structured response contains `rust_code`. On the JSON fallback path, the system additionally prepends `You are an expert C to Rust translator.` and the schema-generated formatting instructions.

2. Visible-Test Generation

The Tester receives only the accepted C source. It does not receive the problem statement, Rust translation, sample I/O, or Judge cases.

```
You are the Tester Agent for a behavior-preserving C-to-Rust translation pipeline.

Using only the C source code below, generate a language-agnostic suite of concrete
standard-input cases. The objective is to validate the observable behavior of the
program by exercising its relevant branches, loop boundaries, arithmetic behavior,
indexing behavior, input shapes, and meaningful edge cases that can be inferred from
the source.

Choose how many cases are necessary to exercise the distinct meaningful behaviors
that can be inferred from this program. Do not reduce the number of cases merely to
keep the suite short. Avoid redundant cases, and give every case a clear purpose.
Each case must contain the complete text that should be sent to standard input.

Before returning a case, verify that it supplies a value for every input operation
reached by that case, including reads controlled by input-dependent loops. Preserve
meaningful line boundaries, delimiters, sentinels, and conversion counts exactly as
required by input APIs such as scanf, fgets, and getchar. Each case must enter the
behavior described by its purpose instead of merely causing the program to terminate
cleanly before its main processing logic runs. Do not use or assume a problem
statement. Do not write C tests, Rust tests, shell commands, expected outputs, or
explanations outside the structured response. Do not invent malformed or incomplete
inputs unless the source explicitly handles them. Expected outputs will be produced
separately by executing the accepted C program.
```

```
C source:
{c_code}
```

Structured response:

```
strategy: string
cases:
  - purpose: string
    input: string
```

3. Visible-Test Regeneration

If cases are rejected during deterministic validation or Validator review, the following text is appended to the generation prompt. Approved cases are preserved and the Tester is asked only for the missing replacements.

```
Generate replacement cases only for the rejected candidates described below.
Previously approved cases are preserved; do not repeat or rewrite them. Follow the
requested replacement count.
Replacement guidance: {regeneration_feedback}
```

This can happen at most once per C program. Cases classified as `inconclusive` are discarded rather than replaced, because the source-only evidence cannot support specific repair guidance.

4. Visible-Suite Correctness Review

The Validator reviews the C source, generated inputs, and deterministic execution summary. It does not see Rust, expected outputs, or Judge evidence.

```
You are the Code Validator reviewing a generated visible-test suite for a behavior-
preserving C-to-Rust translation pipeline.

Using only the C source code and generated standard-input cases below, assess each
case independently. Determine whether it is complete and coherent with the input
protocol that can be inferred from the source. Trace the input operations and their
success conditions far enough to confirm that each case reaches the processing
behavior claimed by its purpose. In particular, account for line boundaries,
delimiters, return-value checks, conversion counts in scanf-family calls, input-
dependent counts, and sentinel sequences. A process that exits normally without
satisfying the reads or guard that enter the intended processing is not a valid test
of that behavior.

Return exactly one assessment for every supplied case identifier. Classify a case as
approved when its input and claimed path are supported by the source, invalid when
it is concretely malformed, incomplete, or internally inconsistent, and inconclusive
when validity depends on constraints that cannot be inferred from the source. Never
reject or downgrade one case because another case is defective.

Review input validity only. Do not evaluate whether the suite has enough cases,
covers every branch, exposes every possible translation error, or is sufficiently
diverse. Limited coverage is not a reason to classify a case or suite as invalid or
```

```
inconclusive. The deterministic report records whether each supplied case terminated
normally, whether its output was stable, and whether that output was empty. Empty
output is not automatically invalid, but it requires checking that the case did not
merely bypass the intended processing because an input operation or guard failed.

The purpose describes behavior or a branch that the case is intended to exercise; it
does not claim that every later condition succeeds unless it says so explicitly. Do
not use or assume a problem statement. Do not analyze Rust code, expected outputs,
translation quality, or Judge results. Return concise findings in the required
structured response and do not reveal hidden chain-of-thought.

For each case, provide concise replacement guidance only when its assessment is
invalid or inconclusive. Do not combine cases into a suite-level verdict.

C source:
{c_code}

Generated cases:
{cases}

Deterministic C execution report:
{deterministic_report}
```

Structured response:

```
case_reviews:
  - case_id: string
    assessment: approved | invalid | inconclusive
    diagnosis: string
    regeneration_guidance: string
```

An LLM invocation or parsing error is recorded separately as `review_failed`; it is not converted into a semantic `inconclusive` assessment.

5. Translation-Discrepancy Analysis

This prompt runs only after a differential visible failure where C passed and Rust failed. It verifies the evidence before any semantic repair is allowed.

```
You are a code validation specialist for behavior-preserving C to Rust translation.
Analyze the C source, Rust translation, and differential visible-test report. First
verify that the failing inputs are complete and valid for every input operation
performed by the C source. If an input is malformed, incomplete, or makes the C
oracle depend on undefined or uninitialized data, classify it as
invalid_visible_test and do not recommend changing Rust to accept it. If the
available evidence cannot support either conclusion, classify it as inconclusive.
Otherwise classify it as translation_discrepancy and provide concise behavior-
preserving repair guidance. Identify only supported discrepancies. Do not reveal
hidden chain-of-thought.

C source:
```

```
{c_code}

Rust translation:
{rust_code}

Differential test report:
{test_report_as_json}
```

Structured response:

```
test_assessment: translation_discrepancy | invalid_visible_test | inconclusive
diagnosis: string
repair_guidance: string
semantic_discrepancies: list[string]
```

Only `translation_discrepancy` can send the pipeline to Rust repair.

6. Rust Repair

Compilation diagnostics or confirmed semantic repair guidance are inserted into the same repair template.

```
You are an expert C-to-Rust translator. Repair the Rust translation while preserving
the C program's behavior and intended algorithm.
Failure category: {failure_category}

C source:
{c_code}

Current Rust translation:
{rust_code}

Issues:
{errors}

Return ONLY fixed Rust code without markdown.
```

The model response contains repaired Rust code. The MCP transport also carries token usage metadata for reporting. The configured repair limit determines whether the pipeline may call this prompt again; reaching it proceeds to final Judge observation without another repair.

Inactive Prompt File

`prompts/translation_prompt.txt` is retained in the repository but has no runtime reference in the current implementation. It must not be described as an active experiment prompt unless the code is changed to use it.